

第 11 节 实操任务 3 实现 Planner-Agent

学习目标：Planner 节点作为图入口，根据 `PLANNER_TARGET_COUNT` 环境变量输出三档策略（lite/standard/full）

目标文件：`workflows/planner.py`（新增）+ 修改 `workflows/graph.py`

背景

实操 2 把 6 节点工作流跑通了。但有两处硬编码：

- `route_after_review` 里写死 `iteration >= 3`
- `collector.py` / `organizer.py` / `reviewer.py` 里的默认值

生产系统要能根据输入规模调整策略，数据多时要保守，数据少时要激进。这就是 Planner Agent 的职责，只规划不执行（Plan, don't execute）。

Planner 挂在图的最前面，输出一个 `plan` dict 写入 state，下游节点读取它调整行为。

步骤 1：更新 state.py 添加 plan 字段（8 → 9）

为什么先加这个字段 · planner_node 输出 `{"plan": {...}}` · 下游 collector / organizer / reviewer 通过 `state["plan"]` 读策略。先加字段 · 再写 planner · 类型安全。

在 `workflows/state.py` 的 KBState 顶部加一行（11-2 后是 8 字段 · 加完后 9 字段）：

```
1 class KBState(TypedDict):
2     plan: dict                # ← 11-3 新增 · Planner 输出策略
3     sources: list[dict]
4     analyses: list[dict]
5     articles: list[dict]
6     review_feedback: str
7     review_passed: bool
8     iteration: int
9     needs_human_review: bool  # 11-2 加的
10    cost_tracker: dict
```

更新顶部 docstring 数据流向：

```

1  """
2  数据流向（V3 完整 · 7 节点）：
3      plan → sources → analyses → review –[pass]→ organize → END
4
5          ↓
6          revise → review（循环）
7          ↓[>max]
8          human_flag → END
9  """

```

验证：

```

1  python3 -c "from workflows.state import KBState; print(f'KBState OK, {le
2  # 期望 · KBState OK, 9 个字段

```

步骤 2：创建 planner.py

提示词：

```

1  请帮我编写 workflows/planner.py：
2
3  需求：
4  1. plan_strategy(target_count=None) 函数，根据目标采集量返回策略 dict
5  2. 三档策略：
6     - lite (target<10): per_source_limit=5, relevance_threshold=0.7, max_
7     - standard (10<=target<20): 10, 0.5, 2
8     - full (target>=20): 20, 0.4, 3
9  3. target_count 默认从环境变量 PLANNER_TARGET_COUNT 读取（默认 10）
10 4. planner_node(state) 函数：LangGraph 节点包装，调 plan_strategy 并返回 {"p
11 5. 每个策略 dict 包含 rationale 字段说明"为什么这么选"

```

参考实现： `workflows/planner.py`

```

1  """Planner Agent – 动态规划节点（V3 流水线节点 ①）
2
3  核心原则：只规划不执行（Plan, don't execute）。
4  Planner 的输出写入 state["plan"]，被下游 Collector/Organizer/Reviewer 共同消
5  """
6
7  import os
8
9
10 def plan_strategy(target_count: int | None = None) -> dict:
11     """根据目标采集量选择策略 – 最小可运行 Planner"""
12     if target_count is None:
13         target_count = int(os.getenv("PLANNER_TARGET_COUNT", "10"))
14
15     if target_count >= 20:
16         return {

```

```

17         "strategy": "full",
18         "per_source_limit": 20,
19         "relevance_threshold": 0.4,
20         "max_iterations": 3,
21         "rationale": f"目标 {target_count} 条, 启用深度模式 (质量优先)",
22     }
23     elif target_count >= 10:
24         return {
25             "strategy": "standard",
26             "per_source_limit": 10,
27             "relevance_threshold": 0.5,
28             "max_iterations": 2,
29             "rationale": f"目标 {target_count} 条, 启用标准模式 (平衡)",
30         }
31     else:
32         return {
33             "strategy": "lite",
34             "per_source_limit": 5,
35             "relevance_threshold": 0.7,
36             "max_iterations": 1,
37             "rationale": f"目标 {target_count} 条, 启用精简模式 (成本优先)",
38         }
39
40
41 def planner_node(state: dict) -> dict:
42     """LangGraph 节点: 把策略写入 state["plan"]"""
43     plan = plan_strategy()
44     print(
45         f"[Planner] 策略={plan['strategy']}, 每源={plan['per_source_limit']},
46         f"阈值={plan['relevance_threshold']}, {plan['rationale']}"
47     )
48     return {"plan": plan}
49
50 ---

```

步骤 3: 修改 graph.py 把 plan 挂到入口

```

1  """LangGraph  workflow图 - 第 11 节完整 7 节点版"""
2
3  from langgraph.graph import END, StateGraph
4
5  from workflows.analyzer import analyze_node
6  from workflows.collector import collect_node
7  from workflows.human_flag import human_flag_node
8  from workflows.organizer import organize_node
9  from workflows.planner import planner_node    # ← 新增
10 from workflows.reviewer import review_node
11 from workflows.reviser import revise_node

```

```

12 from workflows.state import KBState
13
14
15 def route_after_review(state: KBState) -> str:
16     """条件路由：读 state["plan"]["max_iterations"], 不再硬编码 3"""
17     plan = state.get("plan", {}) or {}
18     max_iter = int(plan.get("max_iterations", 3))
19     iteration = state.get("iteration", 0)
20
21     if state.get("review_passed", False):
22         return "organize"
23     elif iteration >= max_iter:
24         return "human_flag"
25     else:
26         return "revise"
27
28
29 def build_graph() -> StateGraph:
30     graph = StateGraph(KBState)
31
32     # 【新增】注册 plan 节点
33     graph.add_node("plan", planner_node)
34     graph.add_node("collect", collect_node)
35     graph.add_node("analyze", analyze_node)
36     graph.add_node("review", review_node)
37     graph.add_node("revise", revise_node)
38     graph.add_node("organize", organize_node)
39     graph.add_node("human_flag", human_flag_node)
40
41     # 【新增】plan → collect 边
42     graph.add_edge("plan", "collect")
43     graph.add_edge("collect", "analyze")
44     graph.add_edge("analyze", "review")
45
46     graph.add_conditional_edges(
47         "review",
48         route_after_review,
49         {
50             "organize": "organize",
51             "revise": "revise",
52             "human_flag": "human_flag",
53         },
54     )
55
56     graph.add_edge("revise", "review")
57     graph.add_edge("organize", END)
58     graph.add_edge("human_flag", END)
59

```

```

60     # 【修改】入口从 "collect" 改为 "plan"
61     graph.set_entry_point("plan")
62     return graph
63
64
65 app = build_graph().compile()

```

注意两处关键变化：

1. `set_entry_point("plan")` — 入口改为 plan
2. `route_after_review` 里 `max_iter` 从 `state["plan"]` 读取，不再硬编码

步骤 4: 验证 collector / organizer / reviewer 已读 plan 字段

第 10 节写的 `collector.py` 和 `organizer.py` 当时还没有 plan 字段·没用过 `state.get("plan", {}).get("per_source_limit", 10)` 这种模式。检查一下它们：

```
1 grep -n 'plan.*per_source_limit\|plan.*relevance_threshold' workflows/co
```

应该看到：

- `collector.py` 读 `per_source_limit`
- `organizer.py` 读 `relevance_threshold`

给 AI Coder 的提示词：

```

1 请修改 workflows/collector.py / organizer.py / reviewer.py, 让它们读取 stat
2 - collector · 读 plan.per_source_limit (默认 10) 替代硬编码采集数量
3 - organizer · 读 plan.relevance_threshold (默认 0.5) 替代硬编码过滤阈值
4 - reviewer · 读 plan.max_iterations (默认 3) 作内部兜底
5 每个节点用 · plan = state.get("plan", {}) or {}; limit = int(plan.get("per

```

Reviewer 需要加一行读 `max_iterations` (虽然当前的 `route_after_review` 已经用了它, Reviewer 内部也可以兜底) ——但这不是必须的。

步骤 5: 运行验证

4.1 默认 (standard 策略) · 完整 7 节点端到端

```

1 cd ~/ai-knowledge-base/v3-multi-agent
2 python3 -m workflows.graph

```

期望输出 A · 首次审核通过 (理想路径)：

```

1 [Planner] 策略=standard, 每源=10 条, 阈值=0.5, 目标 10 条, 启用标准模式 (平衡)
2 [Collector] 采集到 10 条原始数据

```

```
3 [Analyzer] 完成 10 条分析
4 [Reviewer] 加权总分: 7.2/10, 通过: True (第 1 次审核)
5 [Organizer] 整理出 10 条知识条目 (准备入库)
6 [Organizer] 已写入 10 篇到磁盘
```

期望输出 B · 首次未过 → reviser 介入 → 第二次通过:

```
1 [Planner] 策略=standard, 每源=10 条, 阈值=0.5, 目标 10 条, 启用标准模式 (平衡)
2 [Collector] 采集到 10 条原始数据
3 [Analyzer] 完成 10 条分析
4 [Reviewer] 加权总分: 6.3/10, 通过: False (第 1 次审核)
5 [Reviser] 定向修改 5 条 analyses (迭代 1)
6 [Reviewer] 加权总分: 7.5/10, 通过: True (第 2 次审核)
7 [Organizer] 整理出 5 条知识条目 (准备入库)
8 [Organizer] 已写入 5 篇到磁盘
```

期望输出 C · 三次未过 → human_flag 兜底:

```
1 [Planner] 策略=standard, 每源=10 条, 阈值=0.5, 目标 10 条, 启用标准模式 (平衡)
2 [Collector] 采集到 10 条原始数据
3 [Analyzer] 完成 10 条分析
4 [Reviewer] 加权总分: 5.2/10, 通过: False (第 1 次审核)
5 [Reviser] 定向修改 5 条 analyses (迭代 1)
6 [Reviewer] 加权总分: 6.0/10, 通过: False (第 2 次审核)
7 [Reviser] 定向修改 5 条 analyses (迭代 2)
8 [Reviewer] 加权总分: 6.8/10, 通过: False (第 3 次审核)
9 [HumanFlag] ⚠ 达到 3 次审核仍未通过, 标记人工介入
10 [HumanFlag] 已保存到 knowledge/pending_review/pending-xxx.json
```

4.2 切换到 lite

```
1 PLANNER_TARGET_COUNT=5 python3 -m workflows.graph
```

期望输出:

```
1 [Planner] 策略=lite, 每源=5 条, 阈值=0.7, 目标 5 条, 启用精简模式 (成本优先)
2 [Collector] 采集到 5 条原始数据
3 [Reviewer] 加权总分: X/10, 通过: Y (第 1 次审核)
4 ...
```

注意 lite 策略下 `max_iterations=1` ——如果首次就不通过, 直接走 `human_flag`。

4.3 切换到 full

```
1 PLANNER_TARGET_COUNT=30 python3 -m workflows.graph
```

期望输出:

```
1 [Planner] 策略=full, 每源=20 条, 阈值=0.4, 目标 30 条, 启用深度模式 (质量优先)
2 [Collector] 采集到 20 条原始数据
3 ...
```

```
4
5 ---
```

步骤 6：验证清单

检查项	期望	实际
workflows/planner.py 存在	是	
plan_strategy(5) 返回 lite	是	
plan_strategy(15) 返回 standard	是	
plan_strategy(30) 返回 full	是	
graph 的 entry_point 是 plan	是	
route_after_review 读 plan.max_iterations	是	
三种策略都能跑通	是	

步骤 7：提交到 Git

```
1 git add workflows/planner.py workflows/graph.py
2 git commit -m "feat: add Planner agent as graph entry with dynamic strat
3
4 ---
```

7 节点最终拓扑



节点	文件	职责
① Planner	workflows/planner.py	动态规划策略
② Collector	workflows/collector.py	数据采集
③ Analyzer	workflows/analyzer.py	LLM 单条分析
④ Reviewer	workflows/reviewer.py	5 维加权审核
⑤ Reviser	workflows/reviser.py	读反馈定向修改
⑥ Organizer	workflows/organizer.py	整理入库（正常终点）
⑦ HumanFlag	workflows/human_flag.py	人工介入（异常终点）

一个Agent对应一个文件，文件名 = Agent 名，一目了然。

完成！ V3 的 7 节点工作流全部搭建完毕，Planner / Reviewer / Reviser / HumanFlag 四个“质量闭环 Agent”全部到位。

下节课（第 12 节）我们继续给系统加成本控制、安全防护、评估测试，让我们的 Agent 从“能跑”升级到“能上生产”。